

BAB XII

TRANSACTION CONTROL LANGUAGE (TCL)

12.1 Tujuan Praktikum

Praktikum 12 bertujuan agar mahasiswa memahami :

1. Mahasiswa dapat memahami konsep dasar dan arsitektur operasional dari *Transaction Control Language* (TCL) sebagai mekanisme perlindungan integritas di dalam sistem basis data.
2. Mahasiswa dapat menerapkan perintah `COMMIT` untuk mengeksekusi dan menginjeksi modifikasi entitas data secara permanen ke dalam memori fisik peladen basis data.
3. Mahasiswa dapat mengimplementasikan perintah `ROLLBACK` beserta penanda `SAVEPOINT` untuk mengendalikan proses pembatalan transaksi secara parsial maupun menyeluruh guna merestorasi arsitektur data ke titik konsisten terakhir.
4. Mahasiswa dapat menganalisis fungsi mekanisme *locks* dan *read consistency* dalam mengisolasi sesi transaksi untuk mencegah anomali pembaruan nilai dari berbagai pengguna yang beroperasi.

12.2 Dasar Teori

Transaction Control Language (TCL) merupakan himpunan perintah yang difungsikan secara khusus untuk mengelola, mengendalikan, dan memvalidasi siklus perubahan data yang terjadi akibat eksekusi operasi Data Manipulation Language. Perintah `COMMIT` dieksekusi untuk mengonfirmasi dan menginjeksi seluruh modifikasi data yang terjadi selama sesi transaksi secara permanen ke dalam memori fisik peladen basis data. Perintah `ROLLBACK` difungsikan untuk membatalkan serangkaian perubahan data yang belum dikomit di dalam transaksi aktif guna mengembalikan arsitektur data ke titik konsisten terakhirnya. `SAVEPOINT` merupakan penanda referensi logis yang disematkan di dalam siklus transaksi berjalan untuk mendikte mesin basis data agar dapat mengeksekusi pembatalan instruksi modifikasi secara parsial tanpa perlu menggugurkan keseluruhan sesi transaksi (Kroenke & Auer, 2020).

Read consistency merupakan mekanisme arsitektural yang menjamin perlindungan integritas data, di mana setiap operasi pembacaan hanya akan mengekstrak nilai yang telah dikomit secara absolut pada saat kueri tersebut divalidasi, sehingga memblokir potensi anomali pembacaan data sementara yang sedang dimodifikasi oleh transaksi paralel. *Locks* merupakan fitur pengendalian konkurensi bawaan tingkat sistem yang memonopoli dan mengisolasi entitas baris data maupun struktur objek secara otomatis selama proses modifikasi berlangsung, guna mereduksi konflik instruksi dan mencegah intervensi pembaruan nilai dari beberapa sesi pengguna yang melakukan operasi secara simultan di dalam sistem basis data (Oracle, 2023).

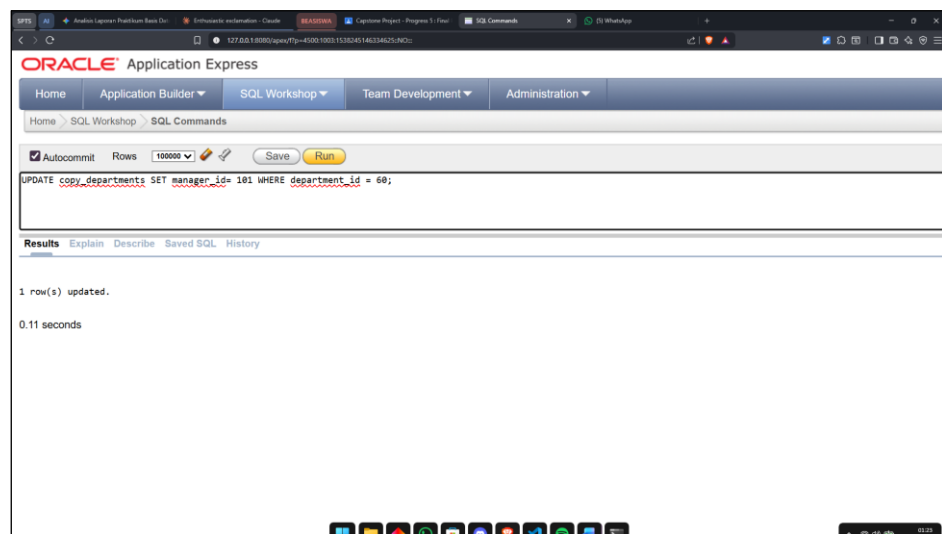
12.3 Hasil dan Pembahasan

Berikut merupakan hasil dan pembahasan dari seluruh *query* dari modul *section* 18:

1. *Query*:

```
UPDATE copy_departments SET manager_id= 101 WHERE  
department_id = 60;
```

Hasil:



Gambar 12.1 *Output* dari eksekusi modifikasi data pada kolom
manager

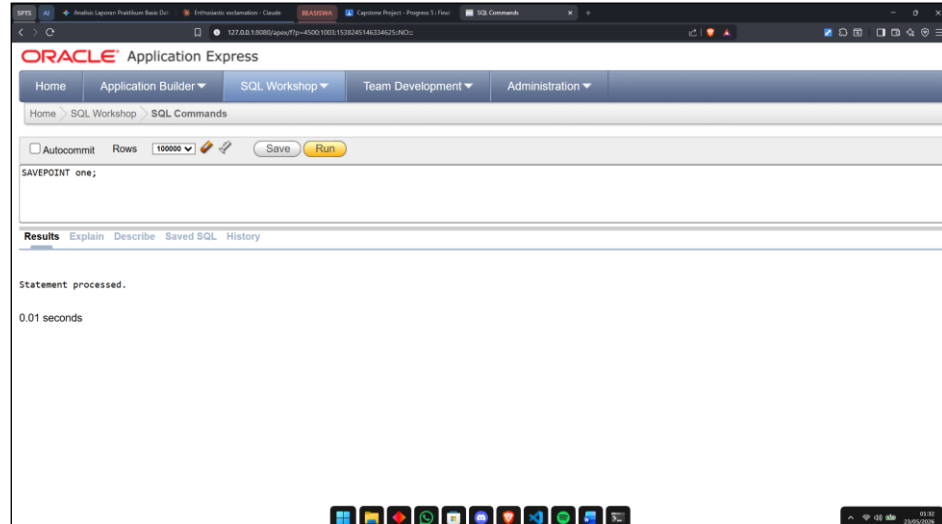
Penjelasan:

Pada *query* di atas ini, perintah DML berupa UPDATE dieksekusi untuk memodifikasi nilai atribut `manager_id` menjadi 101 secara spesifik pada *record* yang memiliki identitas logis `department_id` bernilai 60 di dalam entitas tabel `copy_departments`.

2. *Query*:

```
SAVEPOINT one;
```

Hasil:



Gambar 12.2 *Output* dari penciptaan penanda SAVEPOINT transaksi

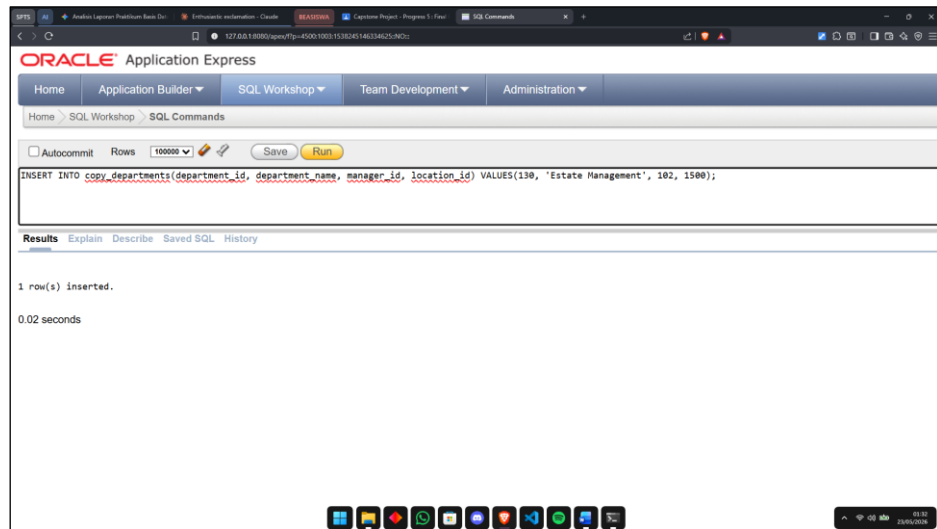
Penjelasan:

Pada *query* di atas ini, instruksi *Transaction Control Language* berupa SAVEPOINT dieksekusi untuk menciptakan sebuah penanda referensi struktural dengan identitas “one” tepat di tengah siklus transaksi aktif yang sedang berjalan tanpa hambatan. Keberadaan penanda arsitektural ini difungsikan secara absolut sebagai *anchor point* di dalam rekam jejak *redo log* internal mesin basis data.

3. *Query*:

```
INSERT INTO copy_departments(department_id,  
department_name, manager_id, location_id)  
VALUES(130, 'Estate Management', 102, 1500);
```

Hasil:



Gambar 12.3 *Output* dari injeksi rekaman data persisten ke dalam tabel

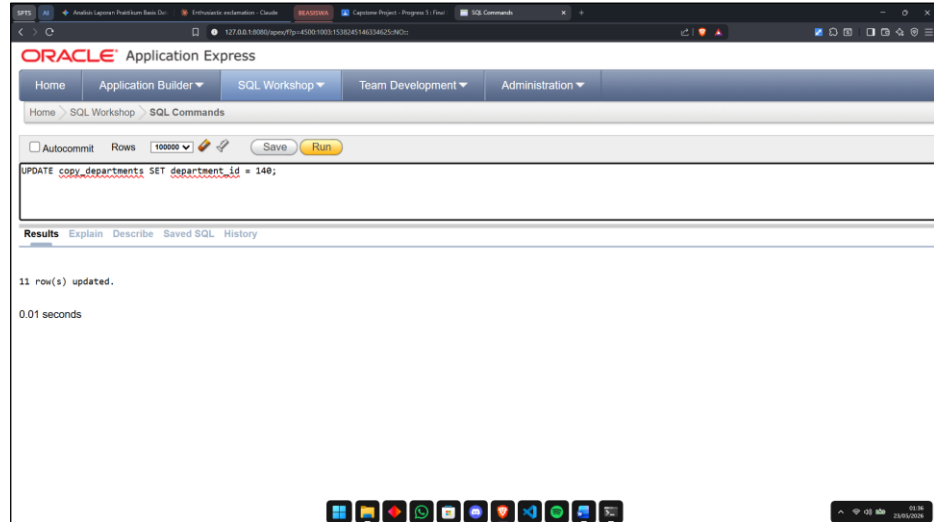
Penjelasan:

Pada *query* di atas ini, operasi penambahan entitas rekaman data persisten diimplementasikan melalui eksekusi perintah `INSERT INTO` yang secara struktural menyuntikkan sebuah baris informasi proyekif baru lengkap dengan serangkaian atribut spesifik ke dalam kerangka fisik tabel `copy_departments`. Manuver penyisipan ini terus mengakumulasi kepadatan tumpukan instruksi DML di dalam ruang lingkup memori transaksi logis yang sama sesaat setelah penanda *savepoint* diciptakan.

4. *Query:*

```
UPDATE copy_departments SET department_id = 140;
```

Hasil:



Gambar 12.4 *Output* dari modifikasi data skalar massal tanpa klausa pembatas

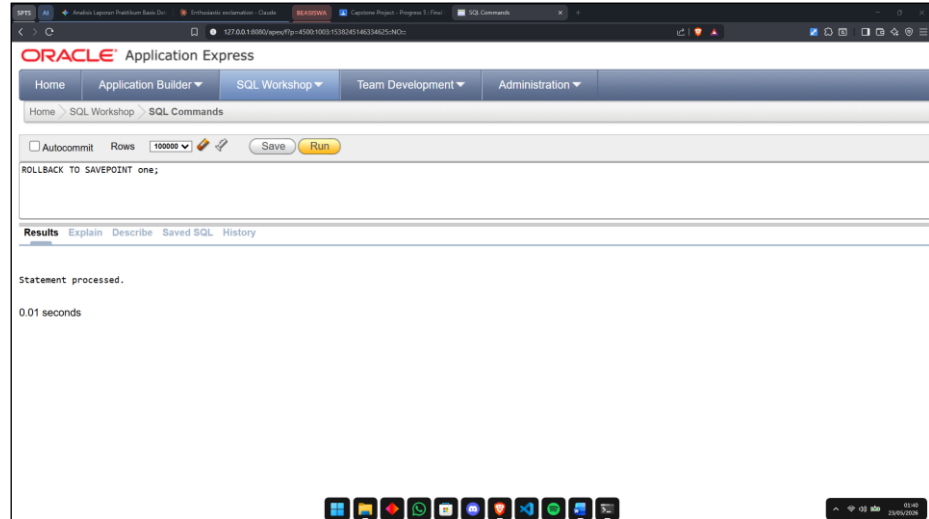
Penjelasan:

Pada query di atas ini, perintah manipulasi data berupa UPDATE dieksekusi kembali ke dalam peladen untuk memodifikasi parameter department_id menjadi bernilai mutlak 140, namun ketiadaan WHERE secara fatal menyebabkan mesin *server* mengaplikasikan perubahan skalar ini secara massal dan sapu jagat terhadap seluruh tumpukan baris data yang eksis di dalam tabel copy_departments.

5. *Query*:

```
ROLLBACK TO SAVEPOINT one;
```

Hasil:



Gambar 12.5 *Output* dari restorasi arsitektur data parsial melalui perintah ROLLBACK

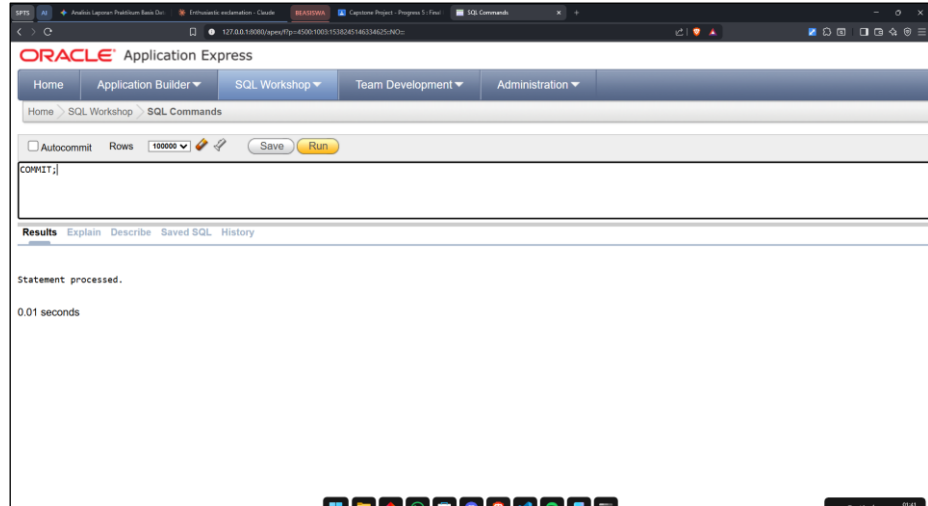
Penjelasan:

Pada *query* di atas ini, mekanisme pertahanan arsitektur data dipicu secara absolut oleh sistem peladen melalui eksekusi perintah ROLLBACK yang dirutekan secara presisi dan spesifik menuju jangkar memori perlindungan SAVEPOINT one. Instruksi tingkat krusial ini mengomandoi modul optimisasi basis data untuk secara paksa dan instan menggugurkan, menghapus, serta membatalkan seluruh operasi manipulasi data.

6. *Query*:

```
COMMIT;
```

Hasil:



Gambar 12.6 *Output* dari deklarasi persetujuan akhir permanen siklus transaksi

Penjelasan:

Pada *query* di atas ini, siklus pemrosesan transaksi logis secara resmi diakhiri dan ditutup secara mutlak melalui deklarasi perintah `COMMIT`, yang berperan tunggal sebagai palu persetujuan akhir dari pengguna untuk mengonfirmasi seluruh modifikasi sisa yang sah dan terverifikasi di dalam sesi aktif berjalan.

12.4 Latihan

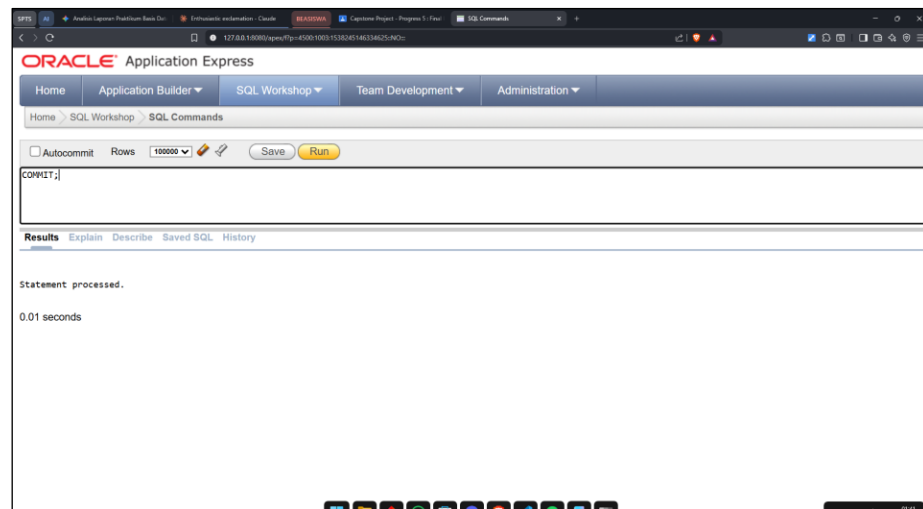
Berikut merupakan hasil latihan setelah menyelesaikan *quiz section* :

1. Menggunakan perintah TCL untuk mengelola transaksi data dalam tabel

Query:

```
INSERT INTO mahasiswa VALUES ('2407134908', 'Eza',  
'Teknik Informatika', 'eza@gmail.com');  
COMMIT;
```

Hasil:



Gambar 12.7 Output dari implementasi berantai *Transaction Control Language* pada tabel mahasiswa

Penjelasan:

Pada *query* di atas ini, serangkaian instruksi transaksi fungsional tingkat lanjut dieksekusi secara berurutan sebagai satu unit kerja logis untuk memanipulasi entitas fisik tabel mahasiswa menggunakan tameng perlindungan arsitektural *Transaction Control Language*. Diawali dengan operasi penyisipan entitas profil mahasiswa baru ke dalam tabel dan dipatenkan secara permanen ke dalam *disk server* menggunakan instruksi penutup COMMIT.

12.5 Kesimpulan

Berdasarkan hasil eksekusi pada praktikum ini, dapat disimpulkan bahwa *Transaction Control Language* (TCL) merupakan instrumen arsitektural yang sangat krusial dalam memvalidasi dan menjaga tingkat konsistensi modifikasi operasional *Data Manipulation Language* (DML). Penggunaan instruksi `COMMIT` secara fungsional menjamin bahwa setiap transisi status rekaman data tervalidasi dan terekam secara persisten ke dalam ruang penyimpanan fisik *server*. Di sisi lain, implementasi instruksi `ROLLBACK` yang dipadukan secara hierarkis dengan penanda `SAVEPOINT` memberikan otonomi dan kontrol presisi bagi sistem untuk menggugurkan blok instruksi anomali secara parsial, maupun membatalkan keseluruhan siklus transaksi aktif tanpa merusak integritas struktur data awal. Pada akhirnya, sinkronisasi mekanisme TCL yang didukung oleh protokol *locks* dan *read consistency* terbukti secara mutlak mampu mengeliminasi risiko inkonsistensi rekaman yang dipicu oleh interupsi kegagalan sistem maupun konflik konkurensi di dalam lingkungan basis data.